MOUNTX SELF-HOSTING

Phase 3: Interpreter

931 lines of Python tree-walking interpreter re-expressed in MountX — then proven byte-identical on the anchor, and finally run three interpreters deep.

product	selfhost/interp.mx - 400 -> 3104 lines
oracle	interpreter.py - 931 lines
the wall	10,400x measured slowdown; the anchor runs in 193 s, not 65 min
the anchor	365 bytes, 23 lines, byte-identical, exit 0, 193 s
the crown	3 layers deep - exit 0, 332 s, 23 bytes
message audit	71 texts, 68 frozen, 3 unreachable, 0 untested

*Every figure in this document is taken from the I-M9a brief, from the three design documents in **selfhost/**, or from a measurement taken while this PDF was generated — a line count, a git tag, or a frozen gate artifact. None of it is from memory. The commit range above is* `first-touching-commit..HEAD` *for this phase's own product file, so* `git log` *over it reproduces the phase's history.*

commit range	2e9830ea559d..c293f17
final tag	self-hosted-complete
generated	2026-10-02 against tree c293f17 (2026-10-02)

The tree named above is the tip these figures were measured against — the parent of the commit carrying this PDF. A document cannot record its own hash, so the commit that names it is `i-m9`*, the tag this work is released under.*

2 THE CONTRACT

The governing document is `selfhost/INTERP_DESIGN.md`, by far the largest of the three, and the only one that opened with its own headline correction: **there is no traceback machinery and no MX_TRACEBACK anywhere in this tree**, and printx is a *statement*, not a native.

The wall, measured before anything was designed

measurement	value	how
host parse rate	`3,829 nodes / 0.005 s = 765,800 nodes/s`	fbtest.mx on an instrumented /tmp copy of interpreter.py, counters only, the repository untouched
host visit rate	`249,168 visits / 0.369 s = 675,251 visits/s`	1.48 us per visit. The tour visits 35,000 nodes per printed line, fbtest 10,800 — neither is print-bound.
one MountX layer	`73.4 nodes/s alone; 57 repo-rate`	parse_repo.mx on one file, dump byte-identical to the oracle's. Use 57-73 as the band.
the gap	**`about 10,400x`** `(765,800 / 73.4)`	one extra interpretation layer multiplies the wall clock by that factor, so the deliverable is a harness and a small corpus, not a faster interpreter.
the projection	`65 min planning number, band 57-72 min, red line 130 min`	written before the first run. The anchor came in at **193 s** — 18x faster — so the cost model is recorded as unvalidated rather than the constants rewritten to fit one data point.

*The honest reading of 193 s against 65 min is that **the model was never validated for this workload**: it assumed the port executes the host's instruction stream (it does not), it costed fbtest.mx's parse at 13.6 ms a node while the dominant cost is 22 stdlib modules, and the depth counter refused nothing.*

The decisions: I-D1 - I-D6 (INTERP_DESIGN.md section 14.3)

#	decision	the alternative that lost
`I-D1`	one kind slot, six kinds, and **no throwx anywhere in the port** — a control signal is data, never an exception	let the port's own throwx carry a tag — less code, and unsound, because the tag is guest data and a guest could forge a return out of it
`I-D2`	transcribe both dispatch chains in the host's order	reorder for speed: buys 0.1%, costs the side-by-side audit that makes 104 behaviours checkable
`I-D3`	**MAX_GUEST_DEPTH = 40**: below the cliff of every shape whose ceiling is above 40, and five times fbtest's deepest recursion	the derived 400, and then 100 — both **inert**: a body nesting nineteen expressions cliffs at 66, under 100, and the physical stack then dies inside the port, catchable by a guest tryx
`I-D4`	delegate all 17 natives plus 4 file builtins; the determinism policy lives in the harness	port the effect-free seven — three new bug classes (idiv int-ness, math_floor round-trip, O(n) len) for no measured gain
`I-D5`	stamp bound methods per access, no cache	cache them — would print true for obj.m == obj.m, making the port more consistent than the language
`I-D6`	one tryx, in callNative, with unboxing outside it	let Python exceptions propagate, or wrap the whole call path — the first loses a test, the second lauds port bugs into guest values

Nothing here is a preference: each row is a choice that one of the corpus cases can falsify, which is the only kind of decision worth recording.

The parity contract: what the differential compares, and what it must not

	the rule	why
`P1`	whole stdout, byte for byte, plus the exit code, for every program in the corpus. No digests, no line counts, no summaries.	against a 3,900 s run, comparing every byte costs 0.01%, and losing a bug because a digest said "same" costs the phase
`P2`	written files are compared too, for writex / appendx / exportx / db_open cases	those builtins return a value rather than text, so their bytes never touch stdout. Each case runs in a fresh $TMPDIR — which is why fbtest.mx's scratch path moved there before phase 3 started
`P3`	the 277 is a differential signal, not a pass criterion	the port printing OK - 261 check(s) passed would be a *silent* failure if the harness grepped for OK. It diffs text and never greps, and case I18 is the negative control: a port deliberately broken to skip one check must make the gate red.
`P4`	a port crash is exit 2 with a PORT-CRASH line, never a guest-shaped error	a port bug surfaced as a catchable value would be indistinguishable from a guest error: stdout would print [Runtime] <something> and the differential would report a mismatch instead of a crash (D-5)
`P5`	six environments, six errors, and the five control interactions E1-E5	E1 a throwx caught in the same activation, where the value is never stringified; E2 unwinding across activations, which obliges the port to restore the current-env pointer on *every* exit path because it has no finally; E3 the four signals must not be confusable; E4 a host failure becomes a catchable value, so error text is output the port owes; E5 a port bug must never become a guest value

The frozen constraints

- **The host is not a library.** interpreter.py is 931 lines — five classes and one big tree-walking Interpreter. interp.mx cannot call it, so everything is ported, and the natives are the only place the port may ask the host for help.
- **No new natives, and that survives phase 3 intact.** The port reads the guest's source with readx and imports with readx plus parser.mx.
- **104 behaviours must be transcribed, not reasoned about**: 16 statement kinds, 17 expression kinds, 13 operators, 44 error raise-paths (42 distinct texts) whose wording a catchx can observe, 6 environment constructors, 8 display shapes. The 17 natives and their 29 guards are free, by delegation.
- **Layer 2 is the last real layer.** At 13 minutes per statement, interp.mx parsing its own source at layer 3 is a thirty-six-year operation.
- **Out of scope:** closures, new natives, generator bytecodes, JSON, http_get, REPL semantics, performance work, and any change to lexer.py, parser.py, interpreter.py or main.py — so the oracle and the port cannot drift in the same direction.

3 THE MEASUREMENTS

measure	value
Product	`selfhost/interp.mx, 400 → **3104 lines** (counted in the tree at report time; the phase's own figure is 3,023)`
Oracle	`interpreter.py, 931 lines; main.py, 177 lines`
The wall	`**10,400x** measured slowdown (765,800 nodes/s against 73.4). The anchor was predicted at 65 min and ran in **193 s**.`
Gate - value layer	`7 cases, 262 golden lines; oracle pins 67/67 reproduced by interpreter.display and type_name`
Gate - exec layer	`68 guest programs, 307 golden lines, 239.51 s; stdout *and* exit code compared against the frozen oracle side`
Message audit	`71 texts, 68 frozen, 3 unreachable, **0 untested**`
The anchor	`365 bytes, 23 lines, md5 a3621b5a0a69c6f71d467c459cac4989, byte-identical, exit 0, stderr empty, 193 s`
The crown	`3 layers deep, exit 0, 332 s, 23 bytes — byte-identical to the host`

The nine milestones, and the gate each one bought

milestone	commit	what it added	interp.mx
I-M0	`2e9830e`	value layer: boxes, display, type_name, the kind slot	`400 lines`
I-M1+I-M2	`1785ee9`	calls, loops, natives; 15/16 statements, 15/17 expressions	`2504 lines`
I-M2 review	`9a97b54`	MAX_GUEST_DEPTH 100 -> 40, because 100 was inert too	`2520 lines`
I-M3	`032342a`	the error pass: 44 raise-paths, a 30-program sweep	`2542 lines`
I-M4	`469b167`	the class layer: ClassDef, New, instance/super, Input	`2795 lines`
I-M5	`4cbf565`	the closure pass: loops proven, reach_audit.py measured	`2796 lines`
I-M6 prep	`e882d58`	three withheld statements, one invisible message	`2830 lines`
I-M6	`0ac2d56`	the anchor: interp.mx runs fbtest.mx byte for byte	`2957 lines`
I-M7	`373d319`	the layer boundary: a native answers the HOST's context	`3048 lines`
I-M7.1	`68528f6`	the crown runs, and the second defect class	`3104 lines`

Line counts are read out of git at each commit, not recalled. The file grew from 400 lines of value layer to 3104 — a transcription plus one marked section that is explicitly not a transcription.

The five stages of the gate, in the order they were added

stage	what it compares	why it exists
value layer (I-M0)	the host's rendering of a value beside the port's rendering of the same value, in one output stream	fast, and not differential in the interesting sense: both sides are being asked the same question by the same harness. It catches a wrong box layout before there is a guest to run.
exec layer (I-M1)	5 guest programs run twice — once through main.py, once through selfhost/run_probe.mx — stdout and exit code against a frozen golden	the first real differential. The exit code is a golden field because main.py's uncaught path is a status *and* a line on stdout, which is why interp.mx transcribes that renderer.
I-M2	8 guests, 140 golden lines, plus the bisected depth measurement	replaced a derived ceiling of ~498 with a measured one, and moved the depth constant twice.
I-M3	the 30-program message sweep: 24 texts a guest reaches, 8 pinned by forged nodes, 9 deferred to I-M4, 1 out of corpus	a program that reaches an error path finds things a forged node cannot.
I-M4 / I-M5	the class layer, then the closure pass and reach_audit.py	reach_audit turns "42 messages" into a number the gate recomputes, so the claim is a tool rather than an assertion.

The recursion ceiling, and why 40 is a rule rather than a margin

depth_probe.mx bisects the port's physical cliff on six ladder shapes by lifting the port's own counter with set_max_depth and reading the **exit code** — 0 fits, 1 is a physical overflow, 2 is a guest that failed to parse. The ceiling **falls toward zero as the body gets heavier**, so no constant beats it, and the limit is chosen by a rule: sit below the cliff of every shape that could still finish something.

shape	ceiling (calls)
five-function ring	330
plain ladder	275
nine nested expressions	115
nineteen nested expressions	66
thirty-nine nested expressions	36
MAX_GUEST_DEPTH (shipped)	**40** — and the corpus's deepest ladder is 24, 60% of the limit

*Above the cliff the counter is inert, and the Python RecursionError then unwinds the port past the restore sites with g_depth left high **and catchable by a guest tryx** — a corrupted interpreter state the guest can catch and carry on from. The limit is a refusal, and a refusal has to die loudly. The derived 400 was inert, and so was the first retake at 100: a body that nests nineteen expressions cliffs at 66, under it. A limit at or below 36 would close the heaviest shapes' escape but refuses y-shaped guests between 37 and 66 that the port completes; matching a cliff exactly is a coin flip.*

*A consequence for the corpus: g8's deepest ladder is 24 and its mutual recursion 30 frames, **not** near the limit of 40. It sits in the row where both sides run, because a golden may only freeze that row — at 41 the port refuses where the host runs, so the old lines would have frozen D-4 as though it were agreement. A refusal inside a golden freezes divergence as if it were parity.*

The crown, and the arithmetic that made it a demo

```
$ MOUNTX_STD=&lt;repo&gt;/stdlib python3 main.py selfhost/interp.mx \
        selfhost/interp.mx selfhost/layer3.mx
exit 0 | stderr empty | 332 s | 23 bytes
hello from layer 3!
42                    byte-for-byte identical to main.py layer3.mx
```

Three interpreters deep: the host runs the port, the port runs the port, and the innermost port runs layer3.mx. The honest arithmetic behind why this is a 4-6 statement program run once: at layer 3 one guest statement costs 805 s, and interp.mx parsing its own source (8,000 nodes) is a thirty-six-year operation. That is not a phase-3 failure; it is the wall the phase measured and designed around, and it is why I-M7 is a demo and I-M6 is the gate.

The bootstrap claim, stated three ways because they are usually conflated

claim	true?	evidence
interp.mx can interpret a program that is itself an interpreter	**yes, at layer 3**, for programs of a few statements	I-M7
interp.mx can interpret interp.mx on a real program	**no**, not on this hardware: 13 min per statement, 36 years for its own parse	13.3
the phase demonstrates a fixed point of the *design*	**yes**: the same AST parser.mx emits is consumed by interp.mx, and interp.mx running a MountX program that runs MountX programs is exercised at I-M7	I-M6 + I-M7

The interesting honest version of the third row: at layer 3 the inner port's own natives are still the host's, so interp.mx interpreting interp.mx exercises the port's exec / evalExpr / callFn chain as guest data — the interpreter's own instruction stream passing through the interpreter's own dispatch, with the kind slot nested six levels deep. ***That is the claim worth making, and it needs six statements, not six thousand.***

4 DEFECTS AS LESSONS

Phase 3 has six times phase 2's surface, and it found six defects of **three different kinds**: a transcription omission, an inert constant, an ordering at a boundary, and two things that were not bugs in the port at all. The last two are the most instructive, because **the crown could not have been reached by adding more tests.**

what was caught	how	the fix	the lesson
mk_super collision. interp.mx defines mk_super(inst, klass) and *imports* parser.mx, which defines mk_super(ln) — the Super node builder. A module function's bare call resolves through globals, so the two-arg box stomped the one-arg builder: no guest containing super could parse.	s03_super_expr.mx, a program in the message sweep. The lexer and parser gates never saw it (each loads only its own port); the interp gate never saw it (no prior case wrote super).	The box is renamed mk_superbox — the 35 mk_* builders are parser.mx's, this box is interp.mx's — and I03/I05's rows move with it.	**The hazard AGENTS.md already warned about, committed inside the port that reads the warning.** A module must not name a function the way a module it imports names one.
Depth 100 was inert. The derived constant was 400; the first retake was 100. A body nesting nineteen expressions cliffs at 66, so at 100 the physical stack died first and the RecursionError unwound the port past its restore sites.	depth_probe.mx, bisected on six shapes and read on the exit code. The old "implied host frames per level" column divided 10,000 by the measured ceiling and called the quotient a rate, which is how a ~1,800 frame reality got quoted as 36.4 to three figures.	MAX_GUEST_DEPTH = 40, chosen by a rule rather than a margin, and recorded as I-D3 with the alternative that lost.	**A counter above a cliff is not a conservative limit; it is an inert one** — and a column derived from an assumption is not a measurement.
The one-second green. The anchor command `main.py selfhost/interp.mx fbtest.mx` was a command that ran nothing and exited 0, because interp.mx had no main().	Reading the exit code rather than the text. This is the most dangerous kind of green there is.	A BOOTSTRAP DRIVER, the only non-transcription section of the file, under a heading that says so — the boundary a reviewer can diff in one place. The command is now literally true.	**A gate that passes on a program which did not run is worse than no gate**, because it is trusted. The argv shift in that driver was also found this way: getting it wrong gives "no guest program specified" and exit 1 in one second.

And the second defect class: orderings, not omissions

These two are the reason this section is worth two pages. Neither is a transcription error — both are things the port did in the right order relative to a boundary it had never been run at. No amount of additional message comparison would have found either.

what was caught	how	the fix	the lesson
argv inheritance. A delegated native answers the HOST's context. sys_args is delegated, so the port answered it from a register the *parent* layer filled. At two layers that is correct by accident: only one layer consults it. At three, the inner interp.mx reads its parent's argv, boots the wrong file, and the failure surfaces as Object of type MxInstance is not JSON serializable — because the path it read was not a path.	Running the port at a layer it had never been run at. The two-layer case stayed byte-identical, so every existing test passed.	One slot, one assignment, one ask: if the env holds sys_args, that function is called — the single place in the port where a name steps aside from natives-before-environment. The bootstrap layer defines a real MountX function returning *this* layer's guest args, and run_program uses one `or` to pick it up. s48 pins the agreement.	**A delegated native answers the HOST's context, so a self-hosting layer must inject its own.** The injected body returns the guest's own args, not its path: a guest never sees its own filename, in the host or here, and an injection that included it would be a new divergence invented to fix an old one.
unbox-before-guard. The host guards each native on the RAW argument (isinstance / is_num) and never unboxes, so every message names the type it was handed. The port unboxed first — right for the adapter, wrong here — so an unboxable value became null, the port wrote a message the host never writes, and the signal it set had already aborted the parse.	The crown, instrumented. The smallest failing case is **two** layers, not three, which is what makes it serious. Then s50, the guard audit: all 17 natives handed a Token instance on both sides.	Two changes, because the len type arm alone was not enough. The guard now receives the boxed arguments alongside the unboxed ones, and the adapter's unbox is **quiet** — it yields null and lets the guard own the message — while eval_file_builtin keeps the loud refusal, where refusing rather than writing a lie is the point.	**A message that matches while the program is dead is not a pass.** With the len arm in place, reverting the quiet unbox SURVIVED the entire corpus — every guard row still matched, because the guard overwrote the message. The corpus was green on a defect that still killed the crown. Only a row checking the program was still alive could find it: len([tok]), where the host answers 1 and continues and a loud port aborts before the next printx.

And the two transcriptions that were not capabilities

- **The [Syntax] mislabel.** The driver's parse-failure branch called format_syntax unconditionally, so a *runtime* refusal surfacing during preparation wore a [Syntax] label. main.py gets this distinction free from Python's exception classes; the port has one channel for both and has to ask the message. render_parse_or_runtime now does, following main.py:38-40. s49 pins both arms.
- **Two more arms corrected.** The port's evalExpr else-arm said Unknown expression {t} where the oracle says Unknown expr type: {t}, and :417 (Cannot assign to field of non-instance) was a too_far refusal although a legal program reaches it — let a = 5; a.x = 1. Neither was a new capability; both were a message that had drifted.

*And the three the audit explains rather than excuses: **:488** (super used but this class has no parent — the env never holds a null super, because a parentless method hits :485 first), **:678** (len expects 1 argument — the lexer gives len its own token and the parser rejects len() with an RPAREN error, so the bare form never reaches eval_call), and **:839** (http_get failed — only a live network call reaches it, and a green gate that depends on a network is not a gate; its arity half IS frozen, with no network at all).*

5 THE ARTIFACTS

Two commands, two stages, plus the audit — and the frozen anchor beside them, because a result nobody can re-run is a story rather than a measurement.

```
# stage 1 - the value layer, and the oracle pins
$ python3 selfhost/parity_interp.py --check
oracle pins: 67/67 reproduced by interpreter.display and type_name
PARITY OK (I-M0 value layer): 7 cases, 262 golden lines, 19.12s

# stage 2 - the real differential: guests run twice, stdout AND exit code
PARITY OK (I-M4 exec layer): 68 guests, 307 golden lines, 239.51s

# the negative control, re-verified against 67 written expectations
$ python3 selfhost/parity_interp.py --selfcheck
oracle pins: 67/67 reproduced by interpreter.display and type_name

# the message audit: a tool, not a claim
$ python3 selfhost/reach_audit.py --check
paths: 73   distinct texts: 71   frozen: 68   uncovered: 3
CHECK OK: 71 texts, 3 uncovered and all three explained.
```

The anchor and the crown, as frozen artifacts

file	what it is
`anchor/fbtest.host.out`	python3 main.py fbtest.mx — the oracle, exit 0, stderr empty
`anchor/fbtest.port.out`	python3 main.py selfhost/interp.mx fbtest.mx — the port. The same 365 bytes, md5 a3621b5a0a69c6f71d467c459cac4989, 23 lines, ending OK - 277 check(s) passed.
`anchor/run.meta`	exit=0 wall=193s
`anchor/selfhost.{host,port}.out`	layer 2: interp.mx interpreting interp.mx. exit=0 wall=270s
`anchor/crown.{host,port}.out`	layer 3, the crown: the same 23 bytes, exit=0 wall=332s
`anchor/layer3.mx`	the 4-6 statement guest the crown runs

MOUNTX_STD is not optional for the anchor. Without it the port's base 4 is selfhost/, so D-1 applies in reverse: the oracle finds stdlib/math/mathfull.mx and the port cannot, and the run dies at the first import. That is the documented divergence, not a defect, and supplying base 1 is the fix.

The four divergences, and the one that could not be closed

	divergence	its bound
`D-1`	the module search list, and which modules exist: the not-found message names the oracle's interpreter.py directory where the port names selfhost/	two halves. Message: no case imports a missing module. Search: found by I-M3's sweep — a guest importx io is found by the oracle through stdlib/ and not by the port, so no exec case imports a stdlib module, and every stdlib-reaching run supplies $MOUNTX_STD.
`D-2`	uncaught non-RuntimeError Python errors print [Runtime] TypeError: in the oracle and [Runtime] in the port	the guest-visible half matches (catchx sees str(e) on both sides); the corpus does not exportx an unserialisable value uncaught
`D-3`	a guest that parses at depth (an importx inside recursion) overflows where the host does not	the port's parse ladder is ~4 levels per construct and shares the 10,000 frames. Corpus rule: no importx below depth 4.
`D-4`	guest depth 41 up to the shape's cliff succeeds on the host and fails on the port	a band of 26 to 290 calls, shape-dependent. At and past the cliff the message is identical, and the corpus's deepest ladder is 24 — 60% of the limit, deliberately.
`D-6`	printx(super) inside a method prints the oracle's str(dict) — a ClassDef node repr with a memory address — where the port refuses and dies loudly	**the real exception, and it arrived with I-M0.** It is semantic, and uncloseable in a way none of the others is: the oracle's own half is not reproducible bytes, because the address differs between two runs of the same program. Nothing can be frozen, so its bound is a corpus rule plus a loud refusal instead of a golden line. typeofx(super) = dict IS reproducible and is checked.

Read D-6 with its hole marked in it: a difference only the port has is a defect, and a difference the oracle cannot reproduce either is a D-n with its corpus rule written down beside it. That is the rule that keeps the register honest — and the reason `typeofx(super)` being a dict while `printx(super)` is unreproducible is not an inconsistency in the corpus but the shape of the problem.

Fifteen claims this document corrected, and where each landed

Every one of these was recollection rather than a stale document: DESIGN.md and PARSER_DESIGN.md were checked and contain none of the citations. The rule the phase adopted afterwards is cheap enough to have no excuse: **read the lines before citing them, and count before claiming a number.**

claimed	what is actually true
self.traceback, MX_TRACEBACK, a renderer at main.py:69-79	no traceback machinery exists anywhere; main.py:69-79 is run_file's read block. grep over 85 files: zero hits. The gate was deleted.
printx is a native	it is a **statement kind**, interpreter.py:309
values_equal at :860-891	:533-547
main.py is 148 lines	177 lines
interpreter.py is 932 lines	wc -l says **931**, and the last line has no trailing newline (od -c on the tail)
the anchor calls time_now_ms and needs a clock normaliser	zero clock / os_cwd / os_listdir / sys_args calls; random is seeded with a literal 42. The normalisers were deleted and the policy rewritten.
25 error messages to transcribe	**44 raise-paths / 42 texts**; 29 native guards are free. An AST walk of every RuntimeError call, bucketed by enclosing function.
29 native guards are free because the host raises them	MountX has no splat, so a wrong-arity call never reaches the native to be refused. 17 of the 29 are transcribed; proven by mutation, where the port died indexing a[0] and reported list index out of range where the host says len expects 1 argument.
sys_args is delegated with a one-place argv shift	the port cannot reach the host's argv at all; the driver installs a register the port reads — which is what I-M7 later had to replace with an injection
the port's call ladder is five host frames deep, so the ceiling is ~498	it is **seven** deep, and the ceiling is 275 calls on a plain body, 115 at nine nested expressions, 66 at nineteen, 36 at thirty-nine. The old 2,300-frame figure was 2,300 *calls*: a label error that hid the constant needed to predict the layer-2 budget.

*Three of the first dozen would have produced **passing gates on code that did not exist** (the traceback gate), **a normaliser that hid real divergence** (the clock), or **a representation chosen for a cost the language does not have** (count + dict, when push is arr + [v] anyway). That is the same failure family phase 2's V1 belongs to.*

What changed outside selfhost/, and why it was allowed

- **AGENTS.MD → AGENTS.md.** The references now read AGENTS.md in README.md, in interpreter.py:22's version comment, and in the two design docs. That second one is a change inside a host file — a comment line, no code, no behaviour — so the honest version of "phase 3 touches no host file" is: the port adds no host-file change, one comment line was renamed earlier in the phase. Recorded rather than buried, because the differential's whole value rests on the oracle being untouched.
- **fbtest.mx's scratch file moved to $TMPDIR.** It previously wrote into the repository, because the gates run with cwd=ROOT — which is precisely how phase 3's own differential could have fed a stale file back as input. Verified after the change: stdout byte-identical to the pre-change run, OK - 277 check(s) passed, zero FAILs, and nothing left in the tree.

6 THE CHAIN

12 tags carry the self-hosting work, in the order they were made. The list below is read from `git for-each-ref --sort=creatordate refs/tags` at generation time, with each tag peeled to the commit it points at, so the hashes are the repository's and not this document's.

#	tag	date	commit	what it froze
1	`i-m0-baseline`	2026-10-01	`2e9830ea559d`	MountX I-M0 baseline: interp.mx value layer (10 tags, display, identity stamps, kind slot) certified by 5-gate differential. Lexer 59/46,944 + parser 49/26,501 byte-exact, fbtest 277, 67/67 oracle pins, 13-mutation control battery.
2	`i-m2-shipped`	2026-10-01	`1785ee96ce67`	I-M2 shipped: calls, control flow, value layer, natives; 8-guest exec gate green
3	`i-m2-review`	2026-10-01	`9a97b5453ccc`	MountX I-M2 review (Q1): MAX_GUEST_DEPTH 100 -> 40, because 100 was inert too
4	`i-m3-errors`	2026-10-01	`032342a92e37`	MountX I-M3: the error pass - 30-program message sweep, mk_super collision fix, D-1 search half
5	`i-m4`	2026-10-01	`469b167c4e02`	MountX I-M4: the class layer - ClassDef, New, instance/super DotAccess, Input
6	`i-m5`	2026-10-01	`4cbf5651ba1f`	MountX I-M5: the closure pass - loops proven, sweep measured, not counted
7	`i-m6-prep`	2026-10-01	`e882d58e0b97`	I-M6 prep: close I16/I17/I10, and fix a message the audit could not see
8	`i-m6`	2026-10-01	`0ac2d56830b4`	I-M6: the anchor - interp.mx runs fbtest.mx byte-for-byte
9	`i-m7`	2026-10-02	`373d319a960e`	I-M7: the layer boundary - a delegated native answers the HOST's context
10	`i-m7.1`	2026-10-02	`68528f66f9de`	I-M7.1: the crown runs - and the second defect class, an ORDER not a transcription
11	`self-hosted-complete`	2026-10-02	`68528f66f9de`	I-M7.1: the crown runs - and the second defect class, an ORDER not a transcription
12	`i-m9-docs`	2026-10-02	`40d0b5cc6a19`	I-M9: the MountX documentary, and a gate that keeps it true

Hashes are the first 12 characters of the object each tag peels to. `git rev-parse <tag>` prints the untruncated 40, so any row can be checked against the repository rather than against this document.

Where this phase sits

- **Phase 3** is frozen at the head of the chain, and `self-hosted-complete` carries the phase's claim in full.
- **Phase 1 (lexer)** and **Phase 2 (parser)** have no tags of their own. They are proven continuously by their gates, and their products — `lexer.mx` (731 lines) and `parser.mx` (1386 lines) — are unchanged across every tag in the list.
- **Phase 3 (interpreter)** owns every tag from `i-m0-baseline` onward. Of the 12 tags in the chain, 12 are phase-3 work; the newest is this documentary, released as `i-m9`.

What the chain asserts

Phase 3's claim, in full: **interp.mx running fbtest.mx is byte-identical to the host** — 365 bytes, 23 lines, exit 0, 193 s — and **interp.mx running interp.mx running layer3.mx is byte-identical too**, exit 0, 332 s, three interpreters deep. Neither is a performance claim and neither makes MountX fast; 332 seconds to print 23 bytes is the sound of an interpreter interpreting an interpreter interpreting an interpreter. The value was never speed. It is that a second independent implementation exists, written in the language itself, and can be shown to agree byte for byte. Final tag: self-hosted-complete.

```
$ git --no-pager for-each-ref --sort=creatordate \
      --format='%(refname:short) %(creatordate:short) %(objectname)' \
      refs/tags
  # run at generation time; this is what the table above is
  # (add | cut -c1-12 to print hashes untruncated)
```

MountX I-M9a — documentary phase reports. Generated by `docs/make_phase_pdfs.py` on 2026-10-02 at 13:02, against tree c293f17, for tag `i-m9`.